

Marie Espinosa
Boris Prince
Thomas STECINSKI
Matthéo NAEGELLEN

lien github : <https://github.com/marieesss/RabbitMQ-Calculator>

Introduction :

Nous avons développé un système de calcul distribué en utilisant RabbitMQ pour gérer la communication entre les différents services. Chaque opération mathématique (addition, soustraction, multiplication, division) est traitée par un worker spécifique, qui reçoit les messages via un exchange de type topic. Le producteur, que nous avons construit, envoie régulièrement des opérations à effectuer. Une application dédiée affiche ensuite les résultats renvoyés par les workers. Nous avons dockerisé l'ensemble des composants afin d'assurer un environnement commun, pour tous les membres de l'équipe.

Explications principales :

Nous avons décidé de dockeriser toutes nos applications dont rabbitMQ afin de faciliter le travail en équipe. En effet, cela permettra d'avoir la même configuration facilement sans problèmes d'environnements. Voici un container rabbitMQ :

```
services:
  > Run Service
  rabbitmq:
    image: rabbitmq:3-management-alpine
    container_name: 'rabbitmq'
    environment:
      RABBITMQ_DEFAULT_USER: admin
      RABBITMQ_DEFAULT_PASS: admin
    healthcheck:
      test: ["CMD", "rabbitmqctl", "status"]
      interval: 10s
      timeout: 5s
      retries: 5
    ports:
      - 5672:5672
      - 15672:15672
      - 15692:15692
    volumes:
      - rabbitmq-data:/var/lib/rabbitmq
      - rabbitmq-log:/var/log/rabbitmq
    networks:
      - app_network
```

Nous avons décidé de définir un network pour s'assurer que les applications soient dans le même réseau pour communiquer avec le rabbitMQ. Nous avons aussi décidé de mettre en place un healthcheck, cela est dû au fait que RabbitMQ met du temps à être configuré, les autres containers communiquant avec lui doivent alors attendre que le broker soit prêt à être utilisé.

```

D Run Service
producer:
  build:
    context: producer
    target: builder
  container_name: 'producer'
  depends_on:
    rabbitmq:
      condition: service_healthy
  ports:
    - '8000:8000'
  networks:
    - app_network

```

Utilisation exchange avec topic :

Nous avons utilisé d'utiliser un exchange de type topic afin que le producer puisse envoyer les nombres à utiliser pour l'opération au bon worker sans se soucier du nom de queue, il devra juste utiliser la bonne routing key.

A) Producer

Le Producer a pour mission d'envoyer aux workers les opérations à effectuer. Marie s'est chargé de la mise en place du producer. Nous avons utilisé Flask (python) pour que l'utilisateur puisse choisir quel type d'opération choisir sous la méthode POST.

Tout d'abord, nous avons créé une connexion avec le broker RabbitMQ. Nous avons initialisé deux variables globales, cela permettra de bloquer un envoi si ces variables ne sont pas None.

```

connection = None
channel = None

```

Nous utilisons le nom du container docker comme host, le port configuré et les identifiants.

```

# Connection à RabbitMQ
connection = pika.BlockingConnection(
    pika.ConnectionParameters( host='rabbitmq',
                              port=5672, credentials=pika.PlainCredentials("admin", "admin")
                              ))
channel = connection.channel()

```

Nous déclarons un nouvel exchange si celui-ci n'est pas existant, le nom de l'exchange doit être le même que celui des workers. On utilise le paramètre durable pour garder les messages pour ne pas perdre les messages si le worker ne répond pas.

```

# New exchange
channel.exchange_declare(exchange=exchange_name, exchange_type='topic', durable=True)

```

Maintenant que la mise en place est faite, il faut envoyer les opérations. Par défaut on utilise les additions. Nous avons initialisé une variable globale **routing_key**

```
routing_key={"key" : "addition", "routing_key": "operation.add"}
```

Cette variable globale peut être changée en utilisant une requête de méthode POST. On récupère l'opération du body, on vérifie si elle existe dans les types d'opérations souhaitées, chaque opération est liée à une routing key (sauf all car elle envoie à tous les workers mais nous en parlerons plus tard). La variable globale **routing_key** est alors mise à jour.

```
@app.route('/', methods=['POST'])
def get_operations():
    global routing_key
    data = request.get_json()
    operation = data['operation']

    approved_operations = [
        {"key" : "addition", "routing_key": "operation.add"},
        {"key" : "soustraction", "routing_key": "operation.sub"},
        {"key" : "division", "routing_key": "operation.div"},
        {"key" : "multiplication", "routing_key": "operation.mul"},
        {"key" : "all"}
    ]

    # Chercher l'opération dans la liste
    operation_found = None
    for op in approved_operations:
        if op['key'] == operation:
            # Change global routine key
            routing_key = op
            operation_found = op
            break

    if operation_found:
        return jsonify({"message": f"Operation '{operation}' found!"})
    else:
        return jsonify({"message": f"Operation '{operation}' not found!"}), 404
```

Pour permettre d'envoyer les opérations au bon worker via l'échange **calc_exchange**, dans l'amélioration numéro 1 et 2 (visible sur la branche feature/producer) on déclare une fonction qui appelle la fonction **send_numbers** toutes les 5 secondes.

Initialisation thread :

```
if __name__ == '__main__':
    init_rabbitmq()
    x = threading.Thread(target=checker_thread)
    x.start()
    app.run(host='0.0.0.0', port=8000)
```

La fonction est appelée toute les 5 secondes

```

channel.basic
def checker_thread():
    while True:
        send_numbers()
        time.sleep(5)

```

On génère deux nombres aléatoirement

```

# Random numbers
num1 = random.randint(0, 100)
num2 = random.randint(0, 100)

```

Si la variable globale channel n'est pas None donc si l'application communique bien avec le RabbitMQ, on peut créer le message à envoyer.

```

# If channel exists, send number to routing key choosen
if channel is not None:
    message=json.dumps({"n1" : num1, "n2" : num2})

```

Puis on publie le message à l'échange **calc_exchange** utilisant la variable globale **routing_key**.

```

channel.basic_publish(exchange=exchange_name,routing_key=routing_key.get('routing_key'), body=message)

```

B) Client

1. Connexion à rabbitMQ

Dans un premier temps, il faut connecter les workers à l'hôte rabbitMQ. Matthéo, Boris et Thomas se sont chargés de la création et de la configuration des workers des opérations. Pour les workers construits avec Python nous utilisons Pika :

```

# Connexion à RabbitMQ
credentials = pika.PlainCredentials('admin', 'admin')
parameters = pika.ConnectionParameters('rabbitmq', 5672, '/', credentials)
connection = pika.BlockingConnection(parameters)
channel = connection.channel()

```

Pour ceux en Js nous nous connectons en utilisant amqplib :

```

1 const amqplib = require('amqplib');
2
3 const rabbitmq_url = 'amqp://admin:admin@rabbitmq:5672';

```

2. Déclaration des échanges :

On définit **calc_exchange** qui est un **échange de type topic** qui permet de router les messages selon une clé spécifique (ex. operation.sub, operation.add, etc.). Cela permet de

répartir les messages vers les workers correspondant à chaque opération.

```
exchange_name = 'calc_exchange'
result_exchange_name="result_exchange"

channel.exchange_declare(exchange=exchange_name, exchange_type='topic', durable=True)
```

3. Déclaration et bind la file

Chaque worker déclare sa propre file dédiée (par exemple, sub_queue pour la soustraction), puis la bind à calc_exchange avec une clé de routage spécifique(operation.add, operation.sub, operation.div, operation.mul.). Cela permet de recevoir que les messages correspondant à l'opération.

Exemple pour la **sub_queue** :

```
queue_name = 'sub_queue'
routing_key = 'operation.sub'

channel.queue_declare(queue=queue_name)
channel.queue_bind(exchange=exchange_name, queue=queue_name, routing_key=routing_key)
```

4. Traitement des messages

La fonction de traitement des messages (**on_request** ou **consume**) effectue les étapes suivantes :

- Récupère et décode les valeurs n1 et n2 du message JSON.
- Simule un traitement avec un délai aléatoire (5 à 15 secondes).
- Exécute l'opération mathématique correspondante (+, -, *, /).
- Publie le résultat dans l'échange **result_exchange** avec la clé operation.result.
- Envoie un accusé de réception (ack) pour indiquer que le message a été traité avec succès.
- Le traitement intègre un bloc try/except pour gérer les erreurs potentielles lors du traitement ou de la réception du message. Dans ce cas le message d'erreur est affiché dans les logs pour faciliter le débogage.

```

# Callback pour traiter les messages
def on_request(ch, method, properties, body):
    try:
        message = json.loads(body)
        n1 = message['n1']
        n2 = message['n2']

        print(f"[Worker SUB] Reçu : {n1} - {n2}")

        # Simulation d'un calcul complexe
        time.sleep(random.randint(5, 15))

        result = n1 - n2

        response = {
            "n1": n1,
            "n2": n2,
            "op": "sub",
            "result": result
        }

        # Publier le résultat
        channel.basic_publish(
            exchange=result_exchange_name,
            routing_key='operation.result',
            body=json.dumps(response)
        )

        # Accuser réception
        ch.basic_ack(delivery_tag=method.delivery_tag)
        print(f"[Worker SUB] Calcul terminé : {n1} - {n2} = {result}")

    except Exception as e:
        print(f"[Worker SUB] Erreur de traitement : {e}")
        ch.basic_ack(delivery_tag=method.delivery_tag)

```

5. Gestion des flux

Dans la suite de notre traitement nous utilisons, **prefetch_count=1** qui nous garantit qu'un worker ne traite qu'un message à la fois, optimisant ainsi l'utilisation des ressources. Puis nous avons aussi **basic_consume** qui démarre la boucle de consommation des messages de la file concernée.

```
# Ne traiter qu'un seul message à la fois
channel.basic_qos(prefetch_count=1)

# Consommer les messages
channel.basic_consume(queue=queue_name, on_message_callback=on_request)

print("[Worker SUB] En attente de messages... CTRL+C pour quitter")
channel.start_consuming()
```

Voici un exemple de message reçu :

```
2025-04-30 10:21:51 [ADD] Reçu : n1 = 5, n2 = 22
```

Un exemple de message envoyé :

```
2025-04-30 10:21:57 [ADD] Résultat envoyé : 27
```

Utilisation exchange avec fanout pour envoyer à tout les workers :

L'utilisation de l'opération "all" permet à ce que le producer envoie à tous les workers connecté à un exchange via une queue de recevoir l'opération à effectuer. Pour cela, il a fallu créer un nouvel exchange mais de type fanout.

1) Producer

Pour le producer, dans la fonction initialisant rabbitMQ, on crée un nouvel exchange **fanout exchange** de type fanout. Marie s'est chargé du type de la mise en place dans le producer.

```
channel.exchange_declare(exchange=exchange_fanout_name, exchange_type='fanout', durable=True)
```

Ensuite, dans la fonction **send_numbers** appelée toute les 5 secondes (pour l'amélioration 1 et 2, dans le projet final la fonction est appelée manuellement), si la variable globale **routing_key** est égale à "all", on publie dans l'échange **fanout_exchange** sans ajouter de routing_key.

```
if routing_key.get('key') == "all" :
    channel.basic_publish(exchange=exchange_fanout_name, routing_key='', body=message)
```

Voici des logs de différents containers montrant que différents workers ont reçu en même temps le message du producer.

```

worker_mul | [MUL] Reçu : n1 = 51, n2 = 84
worker_div | [DIV] Reçu : n1 = 51, n2 = 84
worker_add | [ADD] Reçu : n1 = 51, n2 = 84

```

2) Client

Nous (Boris, Thomas et Matthéo) avons **fanout_exchange** qui est un **exchange de type fanout** qui diffuse chaque message à **toutes les files liées**, sans tenir compte de la clé de routage. Il est utilisé pour du broadcast, comme envoyer un message à tous les workers simultanément.

```

# Déclarer l'exchange direct pour toutes les opérations
exchange_name = 'calc_exchange'
result_exchange_name = "result_exchange"
exchange_fanout = "fanout_exchange"

channel.exchange_declare(exchange=exchange_name, exchange_type='topic', durable=True)
channel.exchange_declare(exchange=exchange_fanout, exchange_type='fanout', durable=True)

```

Affichage Logs :

Les workers après avoir fait les opérations doivent envoyer les résultats dans une application **worker_logs** qui sert juste à afficher les résultats.

Tout d'abord, Marie a créé un container pour l'application **workers_logs**

```

▷ Run Service
worker_logs:
  build:
    context: worker_logs
  container_name: 'worker_logs'
  depends_on:
    rabbitmq:
      condition: service_healthy
  networks:
    - app_network

```

Ensuite dans cette application en python utilisant Pika, on établit une connexion avec le RabbitMQ (comme déjà montré précédemment). Puis nous avons déclaré un exchange **result_exchange** de type direct, on déclare une queue **result_queue** que l'on associe à une routing key **operation.result**.

```

channel.exchange_declare(exchange=exchange_name, exchange_type='direct', durable=True)

channel.queue_declare(queue=queue_name)
channel.queue_bind(exchange=exchange_name, queue=queue_name, routing_key=routing_key)

```

On consomme cette queue pour recevoir les messages


```
# Consommer les messages
channel.basic_consume(queue=queue_name, on_message_callback=on_request)

print("[Worker SUB] En attente de messages... CTRL+C pour quitter")
channel.start_consuming()
```

Puis on affiche le résultat dans les logs du container. Pour finir, on utilise le “ack” pour indiquer à RabbitMQ que le message a été traité.

```
# Callback pour traiter les messages
def on_request(ch, method, properties, body):
    try:
        # Récupération des variables
        message = json.loads(body)
        n1 = message['n1']
        n2 = message['n2']
        operation = message['op']
        result = message['result']

        print(f"[Worker LOGS] Nouveau Résultat: {n1} {operation} {n2} est égale à {result}")

        # Accuser réception
        ch.basic_ack(delivery_tag=method.delivery_tag)
```

Résultat :

```
2025-04-30 10:55:46 [Worker SUB] En attente de messages... CTRL+C pour quitter
2025-04-30 10:55:47 [Worker LOGS] Nouveau Résultat: 38 add 5 est égale à 43
2025-04-30 10:55:58 [Worker LOGS] Nouveau Résultat: 48 add 72 est égale à 120
```

Interface web (partie projet final) :

Pour l'interface web Thomas a mis en place l'interface web de manière basique avec l'utilisation de html/css et js, ensuite nous avons retiré la génération de valeurs aléatoire pour faire nos différents calculs et nous avons modifié la requête POST du producer. Les indications pour accéder à l'interface graphique sont disponibles sur le README.

Calculatrice

Nombre 1

+

▼

Nombre 2

Envoyer

Historique

ensuite lorsque l'on lance un calcul nous pouvons choisir plusieurs type de calculs :

Calculatrice

Nombre 1

✓ +

-

/

*

Toutes

Nombre 2

le programme attends entre 5 et 15 seconde le résultats :

Calculatrice

Envoyer

Historique

[12:19:06] [Résultat] 5 add 2 = 7

[12:18:51] [Attente] Résultat en cours de traitement...

[12:18:51] [Envoi] 5 add 2 envoyé

Et pour l'exécution de tout les types de calcul :

Calculatrice



Envoyer

Historique

[12:20:09] [Résultat] 10 add 2 = 12

[12:20:06] [Résultat] 10 div 2 = 5

[12:20:03] [Résultat] 10 mul 2 = 20

[12:20:00] [Résultat] 10.0 sub 2.0 = 8.0

[12:19:54] [Attente] Résultat en cours de traitement...

[12:19:54] [Envoi] 10 all 2 envoyé